🤖 AI schreibt deine erste Webseite – Keine Zeile selbst tippen!

Entdecken Sie, wie künstliche Intelligenz Ihnen hilft, Ihre erste Webseite zu erstellen, ohne eine einzige Zeile Code selbst zu schreiben. Lernen Sie HTML, GitHub Pages und CSS kennen – alles mit der Unterstützung von KI!

Jetzt starten | Mehr erfahren



Was ist HTML?

Das Grundgerüst einer Webseite

HTML (Hypertext Markup Language) bildet das Fundament jeder Webseite. Es ist die Sprache, die den Aufbau und die Struktur Ihrer Inhalte definiert.

Wichtige HTML-Tags

Lernen Sie die grundlegenden Tags kennen, wie <html>, <head>, <body>, <h1> bis <h6>, <p>, <a> und <img>. Diese Bausteine ermöglichen es Ihnen, Text, Links und Bilder auf Ihrer Webseite zu strukturieren.

KI generiert HTML-Code

Beispiel-Prompt

"Erstelle eine einfache HTML-Seite mit einer blauen Überschrift, einem Begrüßungstext und einem Button, der eine Nachricht anzeigt."

KI-generierter Code

Die KI liefert Ihnen den vollständigen HTML-Code, den Sie direkt verwenden können. Keine Programmierkenntnisse erforderlich!

Anpassungsmöglichkeiten

Sie können den Prompt jederzeit ändern, um verschiedene Elemente oder Stile für Ihre Webseite zu erhalten.

Code kopieren & Editor nutzen

Code markieren

Wählen Sie den gesamten von der KI generierten HTML-Code aus.

Kopieren

Verwenden Sie die Tastenkombination Strg/Cmd + C, um den Code zu kopieren.

In Visual Studio Code einfügen

Öffnen Sie Visual Studio Code und fügen Sie den Code in eine neue Datei ein.

Als "index.html" speichern

Speichern Sie die Datei unter dem Namen "index.html" in einem leicht zugänglichen Ordner.

HTML-Datei speichern & im Browser testen

1

Speichern

Drücken Sie Strg/Cmd + S, um die HTML-Datei zu speichern.

2

Datei finden

Lokalisieren Sie die gespeicherte "index.html" Datei in Ihrem Dateimanager.

3

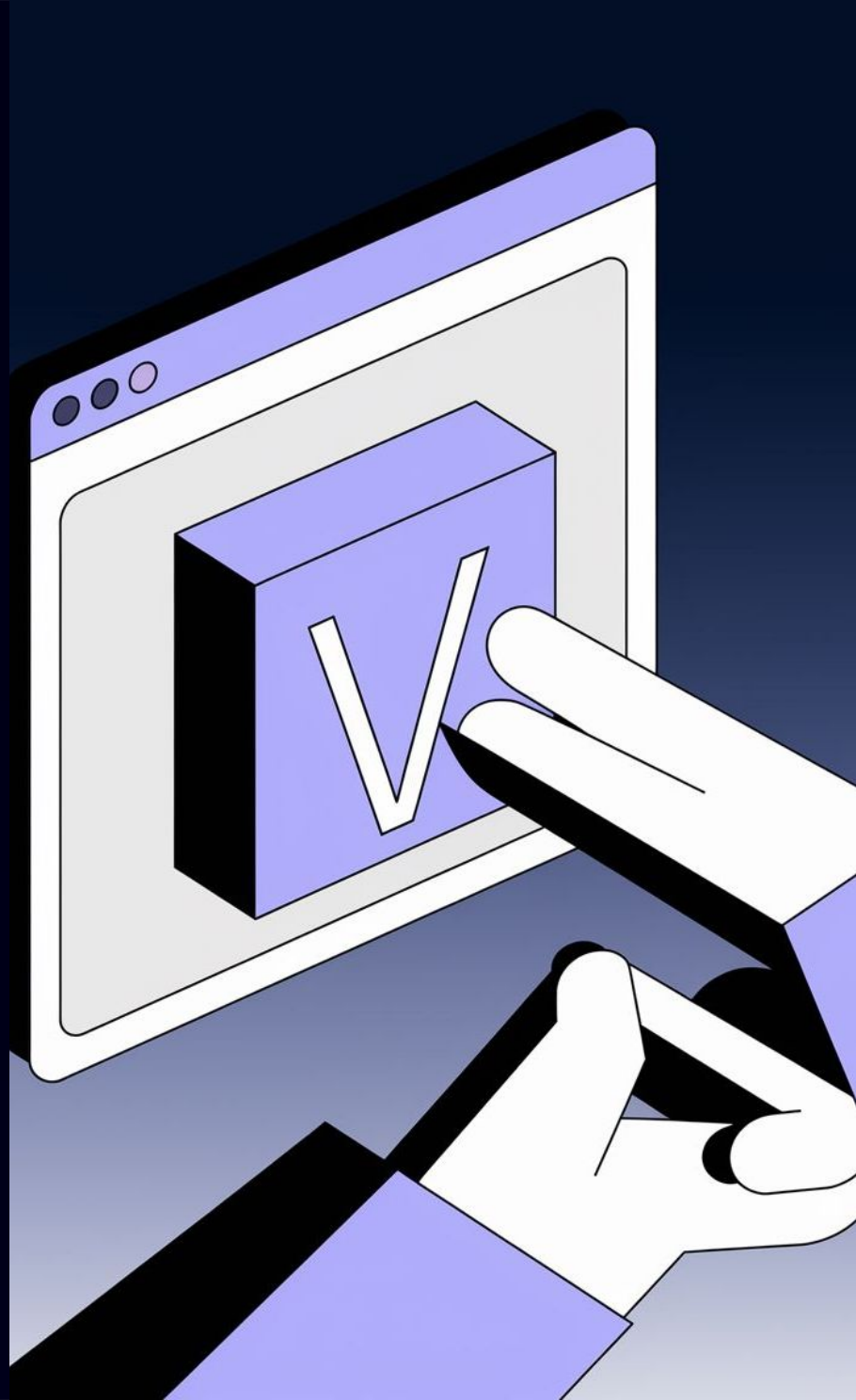
Doppelklick

Öffnen Sie die Datei mit einem Doppelklick in Ihrem Standardbrowser.

4

Überprüfen

Sehen Sie sich das Ergebnis an und notieren Sie eventuelle Probleme oder Verbesserungswünsche.



Fehler finden & KI als Helfer nutzen

Entwicklertools nutzen

Drücken Sie F12 in Ihrem Browser, um die Entwicklertools zu öffnen. Hier können Sie den HTML-Code und mögliche Fehlermeldungen einsehen.

KI um Hilfe bitten

Wenn Sie Probleme entdecken, können Sie die KI um Hilfe bitten. Verwenden Sie einen Prompt wie: "Meine HTML-Seite zeigt nichts an. Bitte überprüfe den Code!"

Zusammenfassung & Ausblick auf Modul 2

1 **Lernziele erreicht**

Sie haben gelernt, wie man mit KI-Unterstützung eine einfache HTML-Seite erstellt, speichert und im Browser testet.

2 **Fehlersuche gemeistert**

Sie wissen nun, wie Sie die Entwicklertools nutzen und die KI bei Problemen um Hilfe bitten können.

3 **Nächstes Modul: GitHub Pages**

Im nächsten Schritt lernen Sie, wie Sie Ihre Webseite kostenlos online stellen können.

🌐 Deine KI-Webseite ist bald online – Kostenlos & ohne Server!

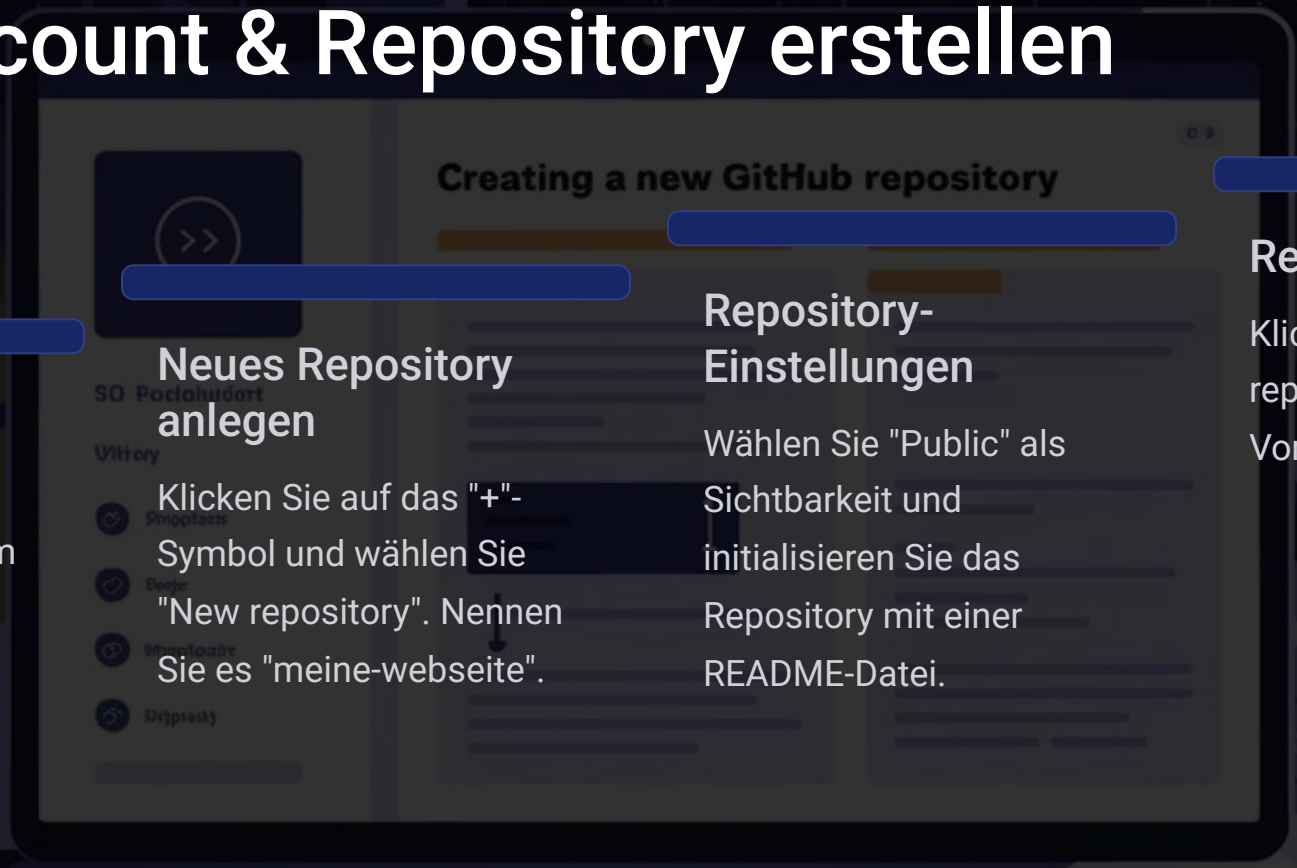
Warum Hosting wichtig ist

Hosting ermöglicht es Ihnen, Ihre Webseite mit der ganzen Welt zu teilen. Es ist der Prozess, bei dem Ihre HTML-Dateien auf einem Server gespeichert werden, sodass sie über das Internet zugänglich sind.

GitHub Pages: Die kostenlose Lösung

GitHub Pages ist ein kostenloser Hosting-Service von GitHub. Er ermöglicht es Ihnen, Ihre Webseite direkt aus Ihrem GitHub-Repository zu veröffentlichen, ohne dass Sie sich um Server oder Hosting-Kosten kümmern müssen.

GitHub Account & Repository erstellen



GitHub Account erstellen

Besuchen Sie github.com und klicken Sie auf "Sign up". Folgen Sie den Anweisungen zur Erstellung Ihres Kontos.

Neues Repository anlegen

Klicken Sie auf das "+"-Symbol und wählen Sie "New repository". Nennen Sie es "meine-webseite".

Repository-Einstellungen

Wählen Sie "Public" als Sichtbarkeit und initialisieren Sie das Repository mit einer README-Datei.

Repository erstellen

Klicken Sie auf "Create repository", um den Vorgang abzuschließen.

HTML-Datei hochladen

1

Repository öffnen

Navigieren Sie zu Ihrem neu erstellten "meine-webseite" Repository auf GitHub.

2

Datei hochladen

Klicken Sie auf "Add file" und wählen Sie "Upload files". Ziehen Sie Ihre index.html per Drag & Drop in den Uploadbereich.

3

Commit-Nachricht hinzufügen

Geben Sie eine kurze Beschreibung für Ihren Upload ein, z.B. "Erste Version meiner Webseite".

4

Änderungen committen

Klicken Sie auf "Commit changes", um den Upload abzuschließen.

GitHub Pages aktivieren

Repository-Einstellungen öffnen

Klicken Sie auf den "Settings" Tab in Ihrem Repository.

Pages-Bereich finden

Scrollen Sie nach unten zum "Pages" Abschnitt in der linken Seitenleiste.

Branch auswählen

Unter "Source", wählen Sie den "main" Branch aus.

Speichern und URL notieren

Klicken Sie auf "Save". GitHub wird Ihnen nun die URL Ihrer live Webseite anzeigen.

Änderungen hochladen & Cache leeren

Änderungen hochladen

Jedes Mal, wenn Sie Änderungen an Ihrer Webseite vornehmen, laden Sie die aktualisierte index.html Datei in Ihr GitHub Repository hoch. Folgen Sie dazu den gleichen Schritten wie beim ersten Upload.

Browser-Cache leeren

Um sicherzustellen, dass Sie die neueste Version Ihrer Seite sehen, leeren Sie den Browser-Cache. Drücken Sie Strg+Shift+R (Windows/Linux) oder Cmd+Shift+R (Mac), um die Seite neu zu laden und den Cache zu umgehen.

Zusammenfassung & Ausblick auf Modul 3

1 GitHub Pages eingerichtet

Sie haben erfolgreich einen GitHub Account erstellt, ein Repository angelegt und Ihre Webseite online gestellt.

2 Kostenlose Hosting-Lösung

Mit GitHub Pages haben Sie eine professionelle und kostenlose Möglichkeit gefunden, Ihre Webseite zu hosten.

3 Nächstes Modul: CSS einbinden

Im nächsten Schritt lernen Sie, wie Sie mit CSS das Design Ihrer Webseite verbessern können – direkt im HTML-Code!



Zusammenfassung & Ausblick auf Modul 3

1 GitHub Pages eingerichtet

Sie haben erfolgreich einen GitHub Account erstellt, ein Repository angelegt und Ihre Webseite online gestellt.

2 Kostenlose Hosting-Lösung

Mit GitHub Pages haben Sie eine professionelle und kostenlose Möglichkeit gefunden, Ihre Webseite zu hosten.

3 Nächstes Modul: CSS einbinden

Im nächsten Schritt lernen Sie, wie Sie mit CSS das Design Ihrer Webseite verbessern können – direkt im HTML-Code!



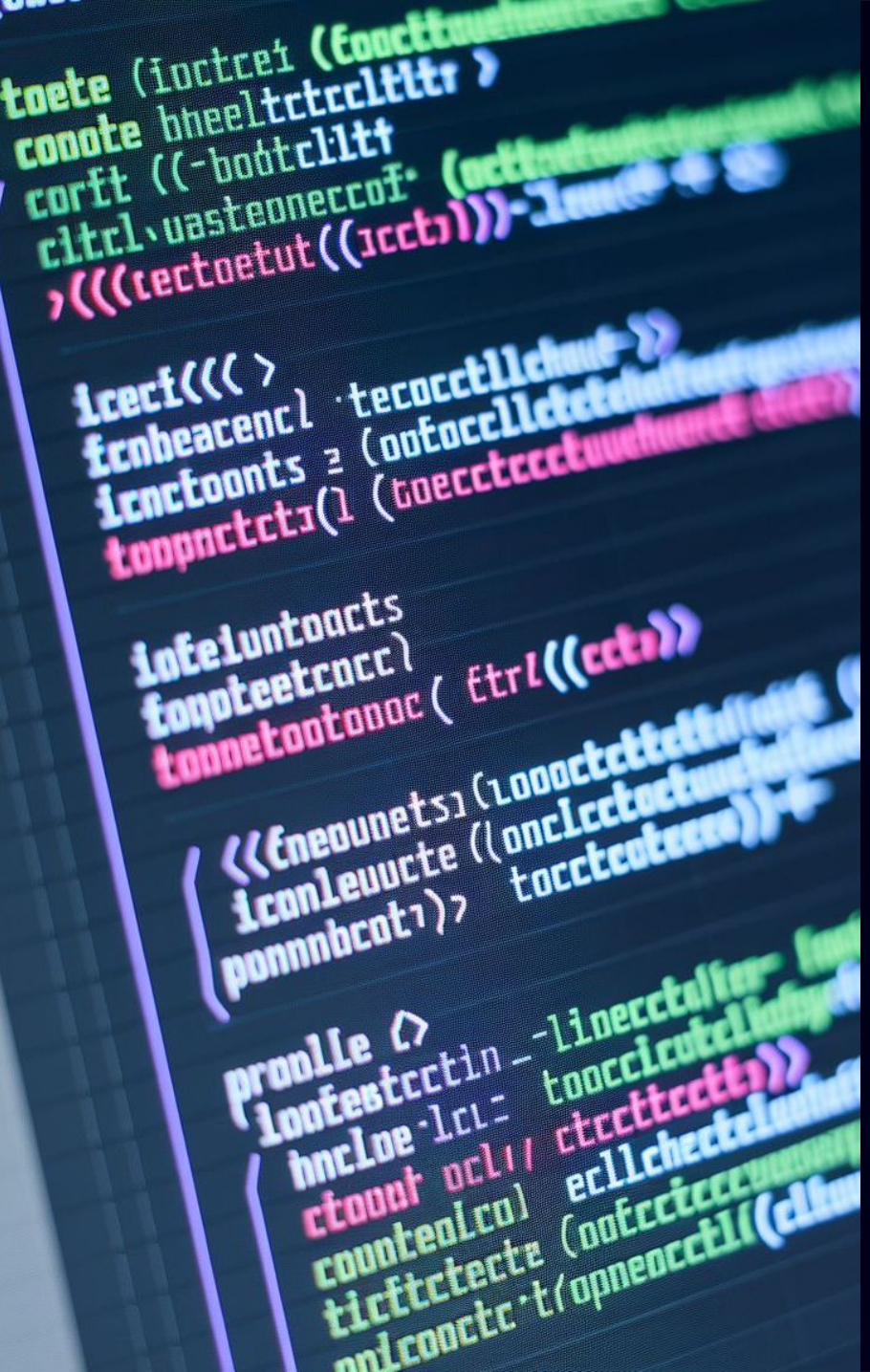
🎨 KI-gestütztes Webdesign – Kein Coding, nur Copy & Paste!

Was ist CSS?

CSS (Cascading Style Sheets) ist die Sprache, die das Aussehen Ihrer HTML-Elemente bestimmt. Mit CSS können Sie Farben, Schriftarten, Layouts und mehr festlegen.

Warum CSS wichtig ist

CSS trennt das Design vom Inhalt, macht Ihre Webseite ansprechender und ermöglicht eine konsistente Gestaltung über mehrere Seiten hinweg.



CSS direkt im HTML einfügen

Style-Tag öffnen

Fügen Sie ein <style>-Tag im <head>-Bereich Ihrer HTML-Datei ein.

CSS-Regeln schreiben

Innerhalb des <style>-Tags können Sie nun CSS-Regeln definieren.

Selektoren verwenden

Nutzen Sie Selektoren wie "body", "h1", ".class" oder "#id", um Elemente anzusprechen.

Eigenschaften festlegen

Definieren Sie Eigenschaften wie "color", "font-size" oder "background-color".

KI-generiertes CSS für schnelles Styling

Prompt für CSS

Bitten Sie die KI um Hilfe beim Styling, z.B.: "Erstelle CSS für eine moderne Webseite mit blauem Farbschema und serifenloser Schrift."

CSS kopieren und einfügen

Kopieren Sie den von der KI generierten CSS-Code und fügen Sie ihn in das <style>-Tag Ihrer HTML-Datei ein.

Grundlegende CSS-Eigenschaften

Farben

Verwenden Sie "color" für Textfarbe und "background-color" für Hintergrundfarbe.

Schriften

"font-family" legt die Schriftart fest, "font-size" die Größe und "font-weight" die Dicke.

Abstände

"margin" für äußere Abstände und "padding" für innere Abstände zwischen Inhalt und Rand.

Rahmen

Mit "border" können Sie Rahmen um Elemente zeichnen und deren Stil festlegen.

CSS-Klassen und IDs

CSS-Klassen

Klassen ermöglichen es Ihnen, den gleichen Stil auf mehrere Elemente anzuwenden. Definieren Sie sie mit einem Punkt, z.B. ".button", und verwenden Sie sie im HTML mit class="button".

CSS-IDs

IDs sind einzigartige Bezeichner für einzelne Elemente. Definieren Sie sie mit einem Hashtag, z.B. "#header", und verwenden Sie sie im HTML mit id="header".

Responsive Design mit CSS

Viewport Meta-Tag

Fügen Sie <meta name="viewport" content="width=device-width, initial-scale=1"> im <head> ein.

Flexible Einheiten

Verwenden Sie relative Einheiten wie % oder em statt fester Pixel-Werte.

Media Queries

Nutzen Sie @media-Regeln, um Stile für verschiedene Bildschirmgrößen anzupassen.

Flexbox oder Grid

Verwenden Sie moderne CSS-Layouttechniken für flexible Anordnungen.

CSS-Frameworks: Bootstrap einbinden

1

Bootstrap CDN einbinden

Fügen Sie den Bootstrap CSS-Link im <head> Ihrer HTML-Datei ein.

2

Container erstellen

Umschließen Sie Ihren Inhalt mit einem div mit der Klasse "container".

3

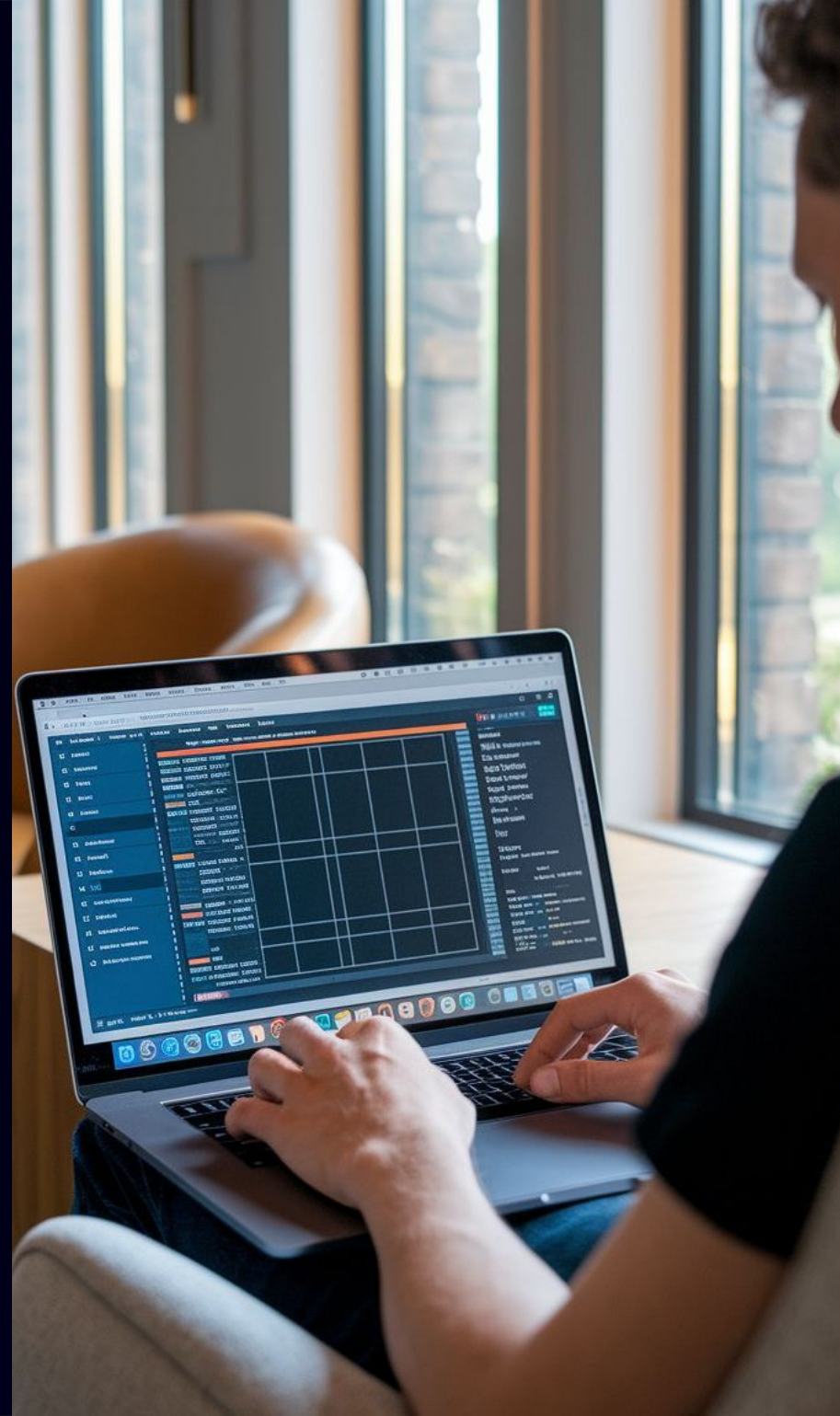
Grid-System nutzen

Verwenden Sie die Klassen "row" und "col" für responsive Layouts.

4

Komponenten hinzufügen

Nutzen Sie vorgefertigte Bootstrap-Komponenten wie Buttons oder Navbars.



CSS-Animationen und Übergänge

Übergänge (Transitions)

Verwenden Sie die "transition"-Eigenschaft, um sanfte Übergänge zwischen Zuständen zu erzeugen, z.B. beim Hover-Effekt.

Animationen

Nutzen Sie @keyframes und die "animation"-Eigenschaft, um komplexere Bewegungen und Effekte zu erstellen.

CSS-Pseudoklassen und Pseudoelemente

Pseudoklassen

:hover, :active, :focus für interaktive Effekte

Strukturelle Pseudoklassen

:first-child, :last-child, :nth-child() für spezifische Auswahl

Pseudoelemente

::before und ::after zum Hinzufügen von Inhalten

Formular-Pseudoklassen

:checked, :disabled für Formularstilisierung

CSS-Spezifität und Kaskade



CSS-Flexbox Layout

Flexbox-Container

Verwenden Sie "display: flex" auf dem Elternelement, um ein Flexbox-Layout zu erstellen.

Flex-Eigenschaften

Nutzen Sie Eigenschaften wie "flex-direction", "justify-content" und "align-items", um die Anordnung der Flex-Elemente zu steuern.

CSS-Grid Layout

Grid-Container definieren

Verwenden Sie "display: grid" auf dem Elternelement.

Spalten und Zeilen festlegen

Nutzen Sie "grid-template-columns" und "grid-template-rows".

Grid-Bereiche definieren

Verwenden Sie "grid-template-areas" für benannte Bereiche.

Elemente platzieren

Positionieren Sie Elemente mit "grid-column" und "grid-row".

CSS-Variablen (Custom Properties)

Variablen definieren

Erstellen Sie Variablen im :root-Selektor mit "--variablenname: wert;".

Variablen verwenden

Nutzen Sie Variablen mit "var(--variablenname)" in Ihren CSS-Regeln.



CSS-Präprozessoren: SASS Grundlagen

Verschachtelung

Schreiben Sie verschachtelte Selektoren für bessere Lesbarkeit.

Variablen

Definieren Sie Variablen mit $ für wiederverwendbare Werte.

Mixins

Erstellen Sie wiederverwendbare CSS-Blöcke mit @mixin.

Partials

Teilen Sie Ihren Code in mehrere Dateien auf und importieren Sie sie.



CSS-Optimierung und Best Practices

1 Minimieren Sie CSS

Entfernen Sie unnötige Leerzeichen und Kommentare für schnelleres Laden.

2 Verwenden Sie Shorthand-Eigenschaften

Nutzen Sie Kurzschreibweisen wie "margin: 10px 20px;" statt einzelner Eigenschaften.

3 Vermeiden Sie übermäßige Spezifität

Halten Sie Selektoren so einfach wie möglich für bessere Wartbarkeit.

4 Organisieren Sie Ihren Code

Gruppieren Sie ähnliche Stile und kommentieren Sie Abschnitte für bessere Übersichtlichkeit.

Responsive Bilder mit CSS

max-width: 100%

Verwenden Sie "max-width: 100%" für Bilder, damit sie sich an die Containerbreite anpassen.

object-fit

Nutzen Sie "object-fit" und "object-position", um Bilder intelligent zu beschneiden oder zu skalieren.

CSS für Druckmedien

Druckstilsheet einbinden

Verwenden Sie @media print {} für druckspezifische Stile.

Hintergründe anpassen

Entfernen Sie farbige Hintergründe und passen Sie Kontraste an.

Seitenumbrüche steuern

Nutzen Sie "page-break-before" und "page-break-after" für saubere Umbrüche.

Unnötige Elemente ausblenden

Verwenden Sie "display: none" für Navigationselemente und andere nicht druckrelevante Inhalte.

CSS für Barrierefreiheit

Ausreichender Kontrast

Stellen Sie sicher, dass Text und Hintergrund einen guten Kontrast bieten.

Fokus-Stile

Gestalten Sie deutliche Fokus-Indikatoren für Tastaturnavigation.

Flexible Textgrößen

Verwenden Sie relative Einheiten wie em oder rem für skalierbare Texte.

Aria-Attribute unterstützen

Stylen Sie ARIA-Rollen und -Zustände für bessere Screenreader-Unterstützung.

CSS für Formulare

Einheitliches Styling

Verwenden Sie Reset-Stile, um browserspezifische Unterschiede auszugleichen und ein konsistentes Aussehen zu erreichen.

Fokus-Zustände

Gestalten Sie deutliche Fokus-Stile für eine verbesserte Benutzererfahrung und Barrierefreiheit.

Responsive Anpassung

Nutzen Sie Flexbox oder Grid für flexible, responsive Formular-Layouts.

CSS-Transformationen und 3D-Effekte

2D-Transformationen

Nutzen Sie translate(), rotate(), scale() für 2D-Bewegungen und -Veränderungen.

3D-Transformationen

Verwenden Sie perspective() und rotateX(), rotateY(), rotateZ() für 3D-Effekte.

Transformationsursprung

Steuern Sie den Ausgangspunkt der Transformation mit transform-origin.

Backface-Visibility

Kontrollieren Sie die Sichtbarkeit der Rückseite bei 3D-Rotationen.

CSS für Dark Mode

Media Query einrichten

Verwenden Sie @media (prefers-color-scheme: dark) für Dark Mode Styles.

Farbpalette definieren

Erstellen Sie eine separate Farbpalette für den Dark Mode mit CSS-Variablen.

Kontraste anpassen

Achten Sie auf ausreichende Kontraste zwischen Text und Hintergrund.

Bilder optimieren

Passen Sie Bilder und Icons an den Dark Mode an, z.B. mit Filtern.

CSS-Fallbacks und Cross-Browser-Kompatibilität

Feature-Erkennung

Nutzen Sie @supports, um zu prüfen, ob ein Browser bestimmte CSS-Eigenschaften unterstützt.

Vendor-Präfixe

Verwenden Sie Präfixe wie -webkit-, -moz-, -ms- für experimentelle oder browserspezifische Eigenschaften.

CSS-Methodologien: BEM und SMACSS

BEM (Block Element Modifier)

Strukturieren Sie Ihre Klassen nach dem Schema .block__element--modifier für bessere Lesbarkeit und Wartbarkeit.

SMACSS (Scalable and Modular Architecture for CSS)

Organisieren Sie Ihr CSS in Kategorien: Base, Layout, Module, State und Theme.

Vorteile

Verbesserte Codeorganisation, reduzierte Spezifität und einfachere Zusammenarbeit im Team.

Implementierung

Wählen Sie die Methodik, die am besten zu Ihrem Projekt passt, und bleiben Sie konsistent.

CSS-in-JS: Styled Components

Komponenten-basiertes Styling

Definieren Sie Stile direkt in Ihren JavaScript-Komponenten für eine engere Kopplung von Logik und Design.

Dynamische Stile

Nutzen Sie JavaScript-Variablen und Props, um Stile dynamisch anzupassen.

CSS-Hacks und Tricks

1 **Clearfix**

Verwenden Sie ::after mit clear: both, um das Floating-Problem zu lösen.

2 **Zentrieren von Elementen**

Nutzen Sie display: flex mit justify-content: center und align-items: center für einfaches Zentrieren.

3 **Textabschneidung**

Verwenden Sie text-overflow: ellipsis für elegante Textkürzungen.

4 **Sticky Footer**

Nutzen Sie Flexbox oder Grid, um den Footer am unteren Bildschirmrand zu fixieren.

Performance-Optimierung mit CSS

CSS-Datei minimieren

Entfernen Sie unnötige Leerzeichen und Kommentare.

Critical CSS

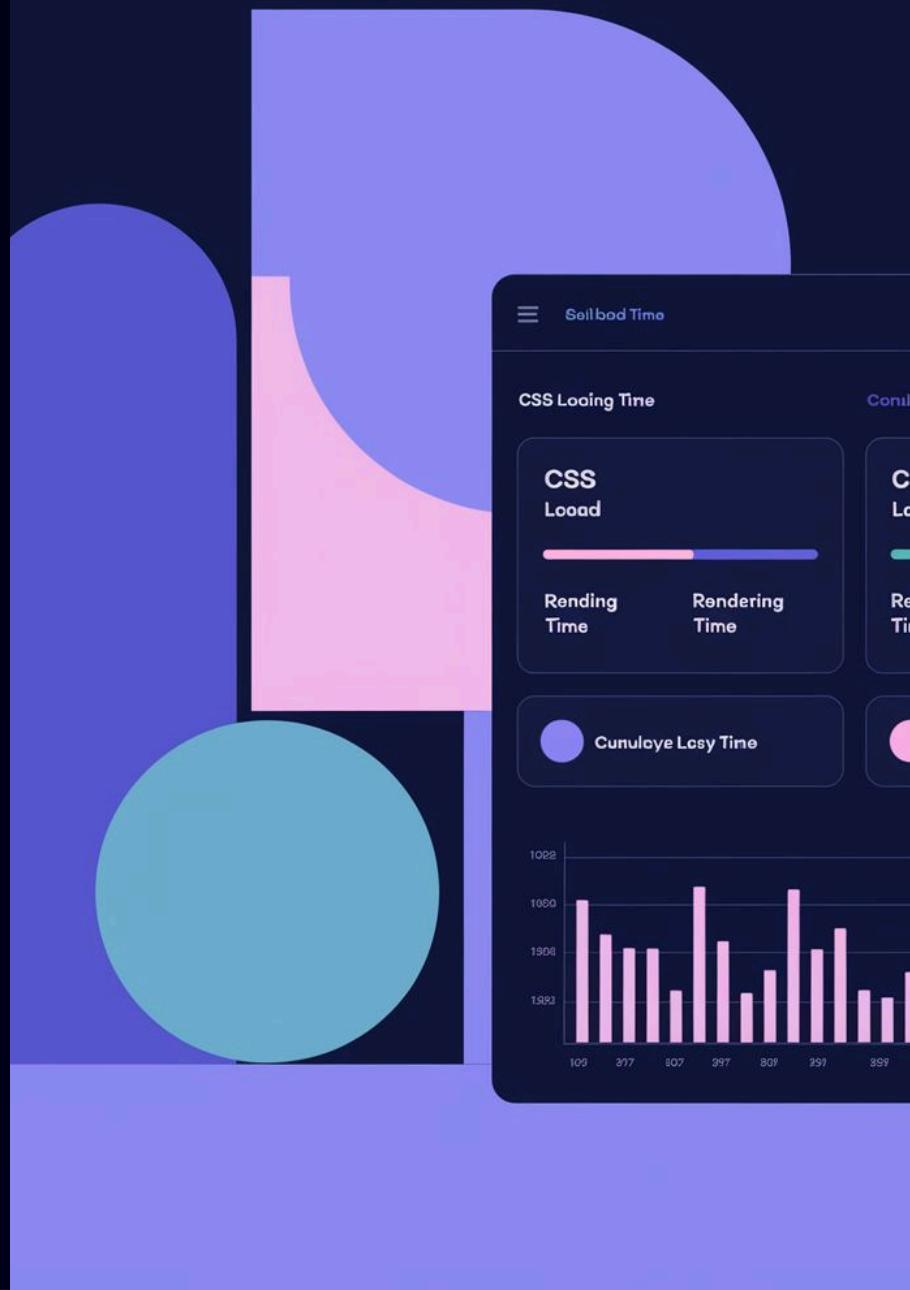
Laden Sie kritische Stile inline und den Rest asynchron.

Selektoren optimieren

Vermeiden Sie übermäßig komplexe Selektoren.

CSS-Sprites verwenden

Kombinieren Sie mehrere Bilder in einer Datei, um HTTP-Anfragen zu reduzieren.



Zusammenfassung und nächste Schritte

1 CSS-Grundlagen gemeistert

Sie haben die wichtigsten CSS-Konzepte und -Techniken kennengelernt.

2 Praxis ist der Schlüssel

Üben Sie regelmäßig und experimentieren Sie mit verschiedenen Stilen und Layouts.

3 Bleiben Sie auf dem Laufenden

CSS entwickelt sich ständig weiter. Verfolgen Sie Blogs und Foren für neue Trends und Techniken.

4 Nächste Schritte

Vertiefen Sie Ihre Kenntnisse in responsivem Design, CSS-Animationen und modernen Layout-Techniken.

